

Normalization & MongoDB Mapping

Inventory / Product Management System — mula sa unnormalized data hanggang 3NF, at pagmapa papunta sa MongoDB collections

1. Unnormalized Data (UNF)

Ito ang hitsura kung isang flat table/spreadsheet lang ang pinaggagamitan ng lahat ng datos — may paulit-ulit na impormasyon (repeating groups) at mixed na entities sa iisang row.

TransactionID	ProductName	SKU	CategoryName	SupplierName	SupplierEmail	SupplierPhone	Qty	UnitPrice	TxnType	TxnQty	TxnDate	PerformedBy
TX001	Bond Paper A4	PAP-001	Office Supplies	ABC Trading	abc@trading.com	0912-345-6789	150	180.00	stock-in	50	2026-09-01	jmark
TX002	Ballpen Black	PEN-010	Office Supplies	ABC Trading	abc@trading.com	0912-345-6789	300	8.50	stock-out	20	2026-09-03	jmark

Problema: paulit-ulit ang Category at Supplier info sa bawat row; hindi atomic ang datos (magkahalo ang Product, Supplier, at Transaction sa iisang table).

2. First Normal Form (1NF)

Tinitiyak na atomic ang bawat value (walang multi-valued fields) at may primary key ang table. Hinihiwalay muna natin ang mga paulit-ulit na grupo papunta sa sariling rows — pero magkadikit pa rin ang lahat ng attributes sa iisang table.

Aksyon: Tiniyak na isang value lang bawat cell (hal. hindi pwedeng "ABC Trading / XYZ Corp" sa isang supplier field). Nabuo na ang unique `TransactionID` bilang primary key.

3. Second Normal Form (2NF)

Dapat 1NF muna, at walang **partial dependency** — ibig sabihin, kung composite key ang ginagamit, dapat umaasa ang bawat non-key column sa BUONG key, hindi lang sa bahagi nito. Sa kaso natin, ang `ProductName`, `SKU`, `CategoryName` ay hindi talaga umaasa sa `TransactionID` — sila mismo ay tungkol sa produkto, hindi sa transaction. Kaya hinihiwalay natin sila sa sarili nilang table:

PRODUCT
<u>SKU</u> (PK), ProductName, CategoryName, Qty, UnitPrice
TRANSACTION
<u>TransactionID</u> (PK), SKU (FK), TxnType, TxnQty, TxnDate, PerformedBy

Nawala na ang partial dependency — malinaw na ang Product info ay "belongs to" sa Product table, hindi sa Transaction.

4. Third Normal Form (3NF)

Dapat 2NF muna, at walang **transitive dependency** — ibig sabihin, walang non-key column na umaasa sa ibang non-key column sa halip na direkta sa primary key. Dito, ang `CategoryName` at ang mga detalye ng Supplier (`SupplierEmail`, `SupplierPhone`) ay hindi direktang umaasa sa `SKU` — umaasa sila sa Category/Supplier, na "by-the-way" lang na naka-attach sa Product. Kaya hinihiwalay pa natin:

CATEGORY
<u>CategoryID</u> (PK), CategoryName, Description
SUPPLIER
<u>SupplierID</u> (PK), SupplierName, ContactPerson, Email, Phone, Address
PRODUCT
<u>SKU</u> (PK), ProductName, CategoryID (FK), SupplierID (FK), Qty, UnitPrice, ReorderLevel
TRANSACTION
<u>TransactionID</u> (PK), SKU (FK), TxnType, TxnQty, TxnDate, PerformedByUserID (FK)
USER
<u>UserID</u> (PK), Username, Email, PasswordHash

Resulta ng 3NF: Limang magkakahiwalay na entity — `Category`, `Supplier`, `Product`, `Transaction`, `User` — walang redundant na datos, at bawat non-key attribute ay umaasa lamang sa primary key ng sarili nitong table.

5. Pagmapa papunta sa MongoDB (Logical Design → Document Model)

Bagama't document database ang MongoDB, ginamit pa rin natin ang normalized na 3NF design bilang batayan. Ang desisyon kung **i-reference** o **i-embed** ay base sa kung paano ginagamit ang datos:

Entity (3NF)	MongoDB Collection	Desisyon	Dahilan
Category	<code>categories</code>	Hiwalay na collection, REFERENCED ng Product	Isang category ay ginagamit ng maraming produkto (shared data). Kung i-e-embed ito sa bawat produkto, mauulit ang parehong impormasyon nang maraming beses.
Supplier	<code>suppliers</code>	Hiwalay na collection, REFERENCED ng Product	Parehong dahilan — iisang supplier ay maaaring nagbibigay ng maraming produkto. Madalas ding nag-uupdate ang supplier info (email, phone) nang hiwalay sa produkto.
Product	<code>products</code>	Pangunahing collection, may <code>category</code> at <code>supplier</code> na Objectid reference	Ito ang "main record" ng system — hindi ito embedded kahit saan dahil kailangan itong ma-query, ma-update, at ma-delete nang mag-isa.
Transaction	<code>stocktransactions</code>	Hiwalay na collection, REFERENCED papunta sa Product at User	Ang mga transaction ay patuloy na dumadami (unbounded growth) — kung i-e-embed ito bilang array sa loob ng Product document, lalaki nang lalaki ang isang document hanggang sa maabot ang 16MB limit ng MongoDB. Mas mainam itong hiwalay na collection na may reference pabalik sa Product.
User	<code>users</code>	Hiwalay na collection, REFERENCED ng	Ang user account ay independent entity na ginagamit din ng authentication system (login/register), kaya dapat itong nakatayo nang mag-isa.

Buod ng Prinsipyo na Ginamit:

- **REFERENCE** kapag: shared ang datos sa maraming records, madalas nagbabago nang hiwalay, o maaaring "unbounded" lumaki (tulad ng transaction logs).
- **EMBED** sana kung: ang datos ay palaging "kasama" lang binabasa ng parent nito, hindi kailanman kinukuha nang mag-isa, at hindi lumalaki nang walang hanggan — wala tayong ganitong kaso sa system na ito kaya lahat ay hiwalay na collections na may reference.

Walang sinadyang denormalization o duplication sa design na ito — sinundan ang 3NF logical model nang direkta papunta sa MongoDB collections gamit ang ObjectId references.